

Comparative Structural and Semantic Evaluation of Fine-Tuned Large Language Models for UML Activity Diagram Generation

1st Rahul Varma U*

Department of Computer Science and Applications
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Amritapuri, India
rahulvarmau@am.amrita.edu

2nd Veena G

Department of Computer Science and Applications
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Amritapuri, India
veenag@am.amrita.edu

3rd Prasanth V Nair

Department of Computer Science and Applications
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Amritapuri, India
prasanthvnair12a@gmail.com

4th Parthip R R

Department of Computer Science and Applications
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Amritapuri, India
parthiprineesh100@gmail.com

Abstract—The manual creation of Unified Modeling Language (UML) Activity Diagrams from natural language software requirements is a time-consuming and error-prone process, leading to the need for automated approaches. Recent large language models (LLMs) have the capability to create activity diagrams from natural language descriptions, but they lack the ability to maintain structural and semantic correctness.

This work explores the application of open-source LLMs for the automatic generation of executable PlantUML activity diagrams from natural language requirements. Three instruction fine-tuned models, namely Qwen 2.5-Coder-7B-Instruct, Llama 3.1-8B-Instruct, and DeepSeek-R1-8B, are tested under baseline conditions and optimized using Supervised Fine-Tuning (SFT) and Direct Preference Optimization (DPO) techniques. A structure-aware semantic evaluation framework is used to compare the generated activity diagrams with ground-truth diagrams based on action coverage, control-flow similarity (CFSim), concurrency coverage, loop coverage, and an aggregate similarity score. The experimental outcome indicates that fine-tuning is an effective method to improve the quality of the diagrams for all models. Among the LLMs tested, DeepSeek-R1-8B performs better in terms of accuracy in representing control-flow, concurrency, and loop constructs. The results clearly show that fine-tuning the models for the specific task leads to a significant improvement in the structural and semantic correctness of the automatically generated UML activity diagrams.

Index Terms—UML Activity Diagram, PlantUML, Large Language Models, Natural Language Processing, Supervised Fine-Tuning, Direct Preference Optimization, Automated Software Modeling, Structural and Semantic Evaluation

I. INTRODUCTION

UML Activity Diagrams are commonly employed to model software behavior. Yet, manually converting natural language software requirements to executable activity diagrams is a time-consuming, error-prone, and domain-specific task that requires modeling expertise [1]–[3].

Generating UML models that are grammatically correct but behaviorally incorrect can result in a chain of errors leading to costly misunderstandings and logical flaws in implementation. Early tools developed for this purpose primarily relied on rule-based natural language processing approaches [1], [4]. Such tools applied syntactic analysis and parts-of-speech tagging to detect

activities and flows of control. However, these methods struggle to cope with linguistic ambiguity and complicated behaviors, such as nested decision-making and parallelism [5].

Recent advances in Large Language Models (LLMs) have enabled the mapping of textual requirements to structural diagrams [6]–[8]. Although advanced multimodal models like GPT-4 and Claude 3.5 demonstrate impressive generalized abilities; they operate as heavily resource-dependent black boxes that often prioritize fluent conversation over proper UML structural correctness [9], [10]. Conversely, localized, open-source parameterized fine-tuning offers maximum flexibility, cost-effectiveness, and structure-based performance assessment. Furthermore, due to API restrictions and highly prohibitive computational costs, a direct experimental comparison with proprietary models was not feasible. Hence, this work focuses exclusively on open-source models to ensure reproducibility. Unlike previous works, we closely integrate fine-tuning with rigorous semantic structure evaluations.

Research Questions: This study investigates the following research questions: (**RQ1**) What effects do SFT and DPO fine-tuning have on the accuracy and structure of LLM-generated UML Activity Diagrams? (**RQ2**) How do baseline and fine-tuned models differ in their ability to represent control flows, parallelism, and looping? (**RQ3**) What is the best-performing LLM among those tested in generating PlantUML activity diagrams?

To answer these research questions, we evaluate three open-source models: Qwen 2.5-Coder-7B-Instruct, Llama 3.1-8B-Instruct, and DeepSeek-R1-8B. We train these models using Supervised Fine-Tuning (SFT) and Direct Preference Optimization (DPO) on a dataset consisting of 275 requirement-PlantUML pairs across 11 different domains. To objectively assess the generated code, we devised a semantic evaluation framework based on control flow similarity, concurrency coverage, and loop modeling accuracy.

TABLE I: Literature Review Summary of Existing Approaches for UML Diagram Generation and Evaluation

Paper	Methodology	Technique Used	Limitation
Critique-Driven Enhancement of LLM Generated Models from Natural Language [9]	Iterative critique-based improvement of LLM-generated activity diagrams	LLM prompting with critique feedback loop	Still produces structural inconsistencies in complex workflows
Automated UML Diagram Evaluation: LLM Assessment versus Teaching Assistants [11]	Comparison of automated LLM grading and human evaluation	LLM-based scoring and educational assessment	Evaluation is subjective and lacks fine-grained structural metrics
Automated Grading Framework for Student Activity Diagrams [12]	Automatic grading system for student activity diagrams	Rule-based UML validation	Cannot handle diverse or ambiguous natural language requirements
Generating UML Sequence Diagrams from Requirements using Large Language Models [8]	LLM-based sequence diagram generation from requirements	Prompt-engineered LLM generation	Limited control-flow accuracy and missing concurrency handling
Text to Diagram Conversion: A Structured Visual Modeling Framework [6]	General framework for converting text to diagrams	Multimodal and LLM-based diagram generation	Not specialized for UML semantics
Code Synthesis from UML Diagram Images via Multimodal Language Models [13]	Converts UML diagram images into code	Multimodal LLM + image-to-text processing	Requires high-quality diagrams and does not verify diagram correctness
Generating Activity Diagrams from Textual Use-Case Specifications [3]	Rule-based transformation of use case text into activity diagrams	Natural language parsing and heuristics	Low scalability and poor handling of complex requirements
A Survey of NLP-Based Approaches for Requirements to UML Transformation [1]	Survey of NLP-based UML generation techniques	Rule-based parsing and ontology mapping	Outdated approaches and lacks LLM-based modeling
Educational Comparison of Fine-Tuned LLMs and Rule-Based NLP for UML Diagram Creation [14]	Educational evaluation comparing fine-tuned LLMs and rule-based NLP for UML diagram generation	Fine-tuned LLM prompting, rule-based NLP extraction	Limited structural correctness verification and lacks detailed semantic validation metrics
Hybrid NLP and Model-Driven Engineering for Automatic Code Generation (NL2Code) [15]	Transformation of natural language and UML models into code	Hybrid NLP + Model Driven Engineering	Focuses on code generation rather than diagram structural correctness

Research Contributions:

The contributions of the proposed research are as follows:

- This research differs from other existing methods used for generating UML diagrams, as they mainly depend on syntactic verification or qualitative analysis. Instead, the proposed framework is based on structural-based semantic evaluation using quantitative metrics such as similarity in control flow, concurrent operation handling, and loop implementation in the generated activity diagrams.
- This research provides a detailed study on the combination of SFT and DPO to enhance the structural accuracy and behavioral correctness of generated UML activity diagrams.
- An extensive comparative study has been done for multiple open-source large language models, which includes ablation studies and cross-dataset validation.

II. RELATED WORK

Automated generation and evaluation of Unified Modeling Language (UML) diagrams from natural language requirements has attracted research attention due to its potential to reduce manual effort, improve consistency, and support software engineering analysis and design activities. Existing research, summarized in Table I, can be broadly categorized into four directions: (A) NLP-based UML generation [3], [4], (B) LLM-based diagram generation [6], [8], [13], [16], (C) Automated UML Assessment and Educational Applications [11], [12], [14], [17], and (D) Behavioral correctness and Verification [10], [18].

A. NLP-Based UML Diagram Generation

Early approaches to UML diagram generation primarily relied on rule-based Natural Language Processing techniques to extract UML elements from textual requirements [1]–[4]. These methods typically employ syntactic parsing, part-of-speech tagging, dependency analysis, and handcrafted heuristics to identify actions, actors, and control structures.

Alashqar proposed a scenario-based approach to creating UML diagrams directly from user requirements, whereas Hnatkowska and Cebinka focused on the extraction of activity diagrams directly from use case descriptions [3]. While these techniques have good interpretability and well-defined behavior, they face problems with linguistic ambiguity, control flow complexity, looping structures, and domain-specific semantics. This results in the generation of incomplete and internally inconsistent diagrams, which may not be fully utilized in other software development domains [5], [19].

B. LLM-Based Diagram Generation

Recent advances in the field of Large Language Models (LLMs) have encouraged the exploration of their application in the field of automated diagram construction using the Unified Modeling Language (UML) [6]–[8], [13], [16]. For instance, Ferrari et al. have shown the effectiveness of LLMs to generate sequence diagrams through natural language requirements [8]. The recent models like GPT-4 and Claude possess excellent generalization abilities [20], but they emphasize fluency rather than strict correctness when generating UML diagrams. On the other hand, the proposed framework makes use of fine-tuned open-source models, which help in controlling the behavioral correctness and cost-effectiveness.

The focus of these previous works remains on generation rather than the correctness of the created diagram. Evaluation methods focus on shallow syntactic aspects of the created diagram or qualitative inspection [11], [12], [17]. Moreover, only a small number of works have explored task-specific fine-tuning and comparison of baseline and fine-tuned LLMs [14].

C. Automated UML Assessment and Educational Applications

Several studies have investigated the use of LLMs for assessing UML diagrams in evaluation settings [11], [12], [14], [17]. Wang et al. evaluated the feasibility of using ChatGPT for grading UML diagrams in classrooms [17], while Fadhlil and Albayati proposed

automated evaluation techniques for student-generated activity diagrams [12]. Boualia *et al.* compared LLM-based grading with instructor evaluations [11].

Nair *et al.* conducted comparative studies between fine-tuned LLMs and rule-based approaches for UML assessment in educational settings [14]. LLM evaluation results are rarely leveraged to improve the generation process itself. Feedback is typically diagnostic rather than corrective, limiting iterative model improvement [10].

D. Behavioral Correctness and Verification

The process of ensuring that the behavior of UML models is correct has been addressed using both formal and execution-based approaches [10], [18]. Ahmed *et al.* developed a framework referred to as MCeT that applies LLM-based reasoning to ensure behavioral correctness of UML models [10]. Siewe and Ngounou focused on execution-based validation of UML activity diagrams to identify behavioral inconsistencies [18]. These approaches are effective in ensuring correctness but largely focus on validation after model creation rather than improving the generation process itself.

Ensuring structural and semantic correctness in generated UML activity diagrams remains a major challenge, particularly for modeling control flow, concurrency, and loops [6], [19]. Large language models often produce syntactically or behaviorally incorrect diagrams, such as missing start or end nodes, faulty branching logic, or improper synchronization in concurrent flows [7]. This highlights the need for generation-aware approaches that guide models to produce structurally and semantically correct activity-flow representations [14]. Apart from techniques specific to UML itself, the generative AI paradigm has also found applications in other aspects of software engineering and even within some domains [15], [21]–[23].

E. Research Gap

Existing studies largely address UML generation, evaluation, and feedback as isolated problems [8], [17]. Rule-based NLP approaches lack semantic robustness [1], [2], while LLM-based methods often prioritize fluency over structural and semantic correctness [6], [7]. However, the impact of fine-tuning the model on the structural accuracy of the generated UML activity diagrams remains to be investigated [14]. Although various automated tools provide valuable feedback [10], [11], [17], this practice is rarely integrated with the generation process. Moreover, only a limited number of studies focus on the generation of PlantUML-based UML Activity Diagrams [7], [14], and they have performed thorough structural and semantic evaluations and comparative studies of the baseline and fine-tuned LLMs on a variety of open-source models.

This paper bridges the gap by evaluating several open-source LLMs for the creation of PlantUML activity diagrams, fine-tuning them using SFT and DPO, and evaluating the results using structure-aware semantic metrics [10], [18]. In contrast to conventional methods like [11], [12] that are either based on syntax checking or human scoring, our method incorporates structure-dependent semantics evaluation by using similarity in the control-flow graph, modeling concurrency, and identifying loop boundaries. Additionally, this research combines fine-tuning with behavior-oriented evaluations, thus providing possibilities for improvements and correctness verification.

III. METHODOLOGY

The present study aims to conduct a structured analysis of the effect of task-oriented fine-tuning on the syntactic and semantic

TABLE II: Structure of Each Dataset Entry

Field	Description
requirement_text	Natural language requirement.
correct_plantuml	Ground-truth PlantUML code.
llm_error_example	Suboptimal PlantUML code.

accuracy of UML activity diagrams produced by large language models. The suggested pipeline involves four steps that should be performed consecutively:

- 1) **Dataset Construction:** Collection of requirement sentences and corresponding ground truth diagrams of PlantUML.
- 2) **Baseline Assessment:** Evaluation of the fundamental model on its basic tasks without adaptation.
- 3) **Dual-Stage Fine-Tuning:** Training on structural mapping using SFT and refinement through behavior reasoning using DPO.
- 4) **Metric Evaluation:** Utilization of our proposed structure-aware framework to measure syntax accuracy against ground truth.

A. Dataset Construction

A diverse benchmark dataset of 275 requirement-PlantUML pairs was developed to associate software requirements with their respective diagrams. Each entry contains a natural language software requirement, the correct UML Activity Diagram expressed in PlantUML, and a suboptimal PlantUML example. The dataset spans a wide range of domains including authentication, banking, business processes, e-commerce, healthcare, inventory management, libraries, social media, food delivery, education, and travel services. Since there is a lack of publicly accessible requirement-PlantUML data at scale, a selected portion of externally curated requirement data was used for testing the validity of generalization. The results were in line with domain independence.

As shown in Fig. 1, the dataset construction began as candidate pairs automatically generated from the Gemini large language model. This process generated both the natural language requirement text and the PlantUML activity diagrams. To mitigate the risk of hallucinations and diagrammatic errors, each example underwent multiple rounds of validation. Independent language models, including ChatGPT, Perplexity, Claude, Grok, and Black-box AI, evaluated whether the diagram correctly represented the underlying behavior, examining for sufficient activity expression, correct control flow, correct loop depiction, and concurrency characteristics such as fork/join. Only examples that passed these tests were retained.

The dataset was checked manually by an expert in UML using a stringent checklist based on reachability and conditional statements. The diagrams that passed both checks were considered authoritative ground-truth annotations.

Each dataset entry contains the fields listed in Table II.

All the PlantUML diagrams were made uniform to maintain a consistent syntax and structure. The dataset was split into 80% training and 20% testing data, where the training data was used for fine-tuning the model, and the test data was held back for evaluating the structural and semantic correctness of the baseline model and the fine-tuned model.

B. Model Selection

As shown in Fig. 2, three open-source instruction-tuned LLMs were tested: **Qwen 2.5-Coder-7B-Instruct**, **Llama 3.1-8B-Instruct**, and **DeepSeek-R1-8B**. They were chosen because they are publicly available, have the ability to produce code, and represent different approaches and goals in their design.

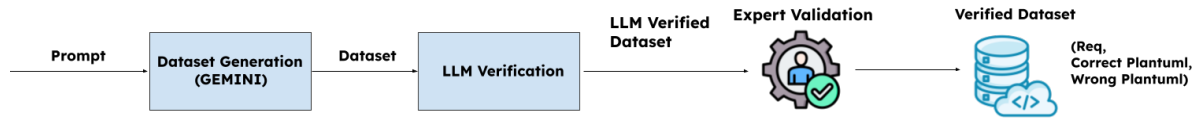


Fig. 1: Dataset construction and verification workflow.

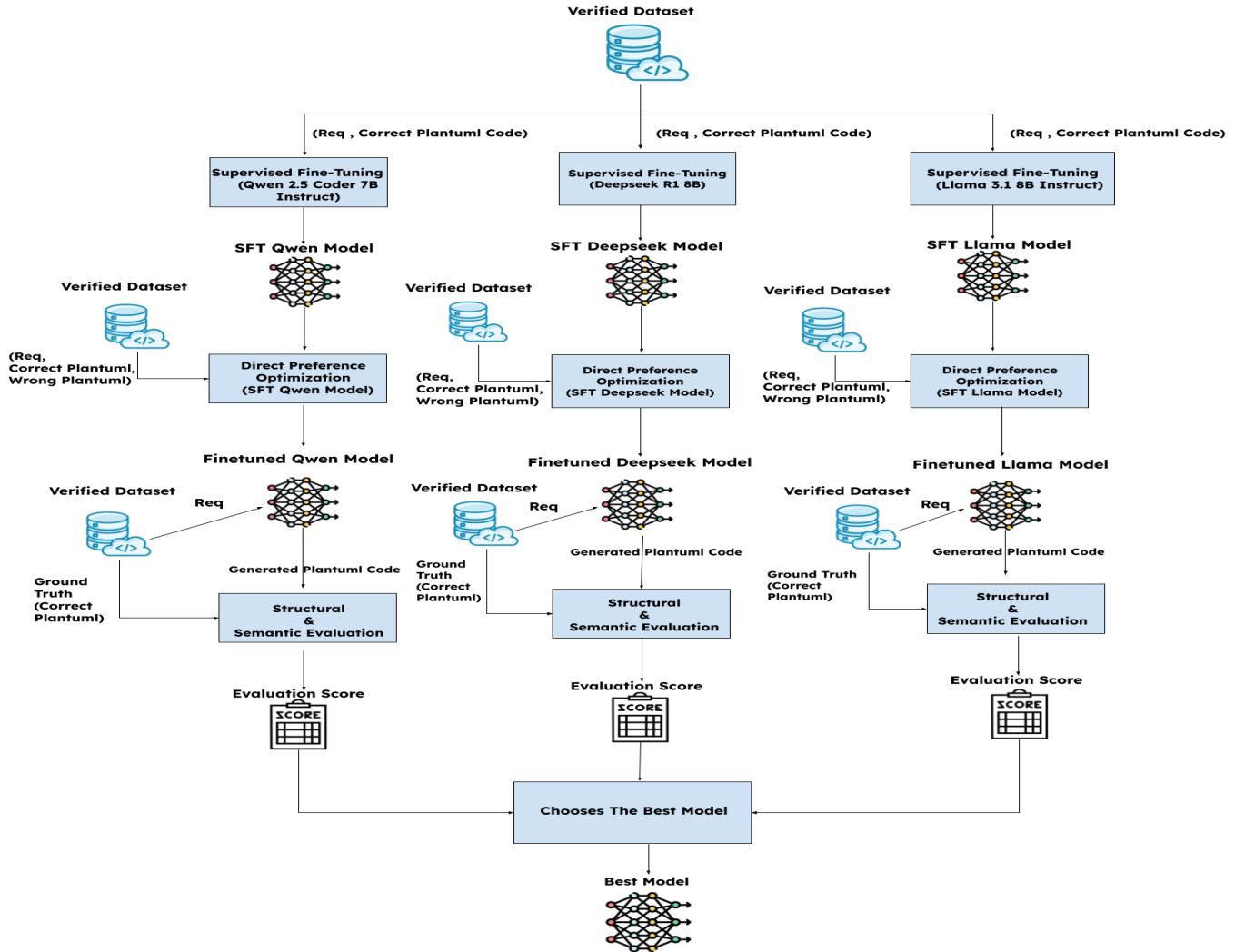


Fig. 2: Model Fine-Tuning and evaluation workflow.

Qwen 2.5-Coder-7B-Instruct is optimized for programming and structured code generation, **Llama 3.1-8B-Instruct** is optimized for general-purpose reasoning, and **DeepSeek-R1-8B** is optimized for logical reasoning with a focus on structured result generation. To ensure fairness and clarity, each model was tested on the same dataset, trained in the same way, and assessed using the same metrics.

C. Supervised Fine-Tuning (SFT)

As shown in Fig. 2, after verifying the baseline, we fine-tuned the models using labeled pairs. Each natural language requirement is paired directly with its corresponding PlantUML activity diagram on the 80% training segment. SFT assists the models in learning typical PlantUML formatting schemas.

Parameter-Efficient Fine-Tuning: To reduce computational complexity and GPU memory requirements, we employed Quantized Low-Rank Adaptation (QLoRA), as shown in Table IV. The weights of the pre-trained model were loaded with 4-bit

quantization and were left frozen, and small trainable low-rank adapter matrices were appended to the transformer attention modules. Only the LoRA adapter weights were updated during training, allowing the model to learn UML structural patterns without sacrificing its general language abilities.

Note that SFT by itself does not ensure full behavioral correctness. Matters such as condition evaluation, loop boundaries, and synchronization of concurrent behaviors might still be incorrect. To handle this, we append a preference-based optimization step to further improve semantic correctness.

D. Direct Preference Optimization (DPO)

As shown in Fig. 2, after SFT, we fine-tuned our models via DPO for better semantic reasoning. Although SFT trains the model to mimic structural syntax, the model sometimes finds it difficult to maintain synchronization. On the other hand, DPO enables the model to differentiate between correct and incorrect execution logic.

A preference function was created for each NL statement where the model is required to learn the difference between a preferred valid diagram and an unacceptable diagram. The model is provided with a greater probability gap for the preferred valid diagram using a pre-defined beta value. This enables the model to make corrections for common mistakes that SFT cannot detect.

E. Training Configuration and Reproducibility

As shown in Table III, all models were fine-tuned in the same environment with the same hyperparameters.

TABLE III: Fine-Tuning Hyperparameters

Parameter	Value
Optimizer	AdamW
Learning Rate (SFT)	2×10^{-4}
Learning Rate (DPO)	5×10^{-6}
Per-Device Batch Size	2
Gradient Accumulation Steps	4
Effective Batch Size	8
Maximum Sequence Length	2048 tokens
Quantization	4-bit (NF4)

The experiments were carried out with the Hugging Face Transformers library. The parameter-efficient adaptation was performed with the PEFT library, and the preference optimization was performed with the TRL library. The quantization was done with the bitsandbytes backend, which allowed the training of 7B-8B models on a single GPU.

QLoRA Configuration:

TABLE IV: QLoRA Adaptation Parameters

Parameter	Value
LoRA Rank (r)	16
LoRA Alpha	16
LoRA Dropout	0.0
Target Modules	Transformer attention projection layers
Bias Training	None
Base Model Precision	4-bit Quantized Weights

In the DPO phase, for each requirement, there were two diagrams: the preferred (ground-truth) diagram and the rejected (incorrect) diagram. The objective increased the preference for the preferred diagram over the rejected diagram. The DPO temperature beta value was 0.1.

F. PlantUML Diagram Generation

For each requirement, the models were asked to generate a UML Activity Diagram in PlantUML notation, limited to a compilable form that starts with @startuml and ends with @enduml. We explicitly selected PlantUML because its textual representation is extremely expressive about actions, decision points, branches, merges, loops, and parallel execution (fork/join), allowing us to automatically identify the underlying structure. All results were verified by compiling the PlantUML code. If the code failed to compile or failed to produce an activity diagram, it was deemed incorrect and assigned a structural similarity score of 0.

With the baseline configuration, the models were producing output diagrams that resembled sequence diagrams, state views, or even natural language pseudocode. However, they were failing to produce the main activity flow components such as start nodes, decisions, and termination nodes. After fine-tuning the models and optimizing the preference scores, the models were producing the correct activity flow components. The PlantUML code was compiled and executed to produce a control flow diagram. The resulting diagram was then compared with the ground-truth

PlantUML code using the structural and semantic evaluation metrics.

G. Structural and Semantic Evaluation Metrics

To determine the level of accuracy in the representation of the behavior and the quality of the UML activity diagrams, we formulated a set of semantic and structural metrics, as shown in Table V.

TABLE V: Summary of semantic and structural evaluation measures.

Measure	Description
ActionCov (Action Coverage)	Required actions present via cosine similarity embedding extraction.
CFSim (Control-Flow Similarity)	Path-based semantic similarity of execution flow using graph alignment.
LoopCov (Loop Coverage)	Accuracy identifying correct backward edges/loop bounds.
ConcCov (Concurrency Coverage)	Correct modeling of parallel execution bounds (fork/join).
Aggregate Score	Weighted aggregated overall score.

The action coverage score is defined based on a semantic F1-score approach, which evaluates recall as how accurately the ground-truth actions appear in the synthesized chart, while evaluating precision as how much the hallucinated actions are penalized. Semantic similarity between actions is determined by calculating embeddings for sentences based on the *all-MiniLM-L6-v2* transformer model.

The calculation of control flow similarity (CFSim) is based on the use of semantic graph alignment through paths. In this method, the ground truth PlantUML diagram and the generated PlantUML diagram are converted to directed graphs in which activities constitute nodes and execution order forms edges. The paths are then obtained from the directed graph using the depth-first search algorithm, after which paths are represented in terms of ordered sequences of activities. Similarity between semantic paths is calculated using sentence embedding, with each of the ground truth paths compared with one or more generated paths and the best match identified.

The diagram quality score R , termed the Aggregate or Overall Score in the results tables, is calculated as a weighted sum of the structural and semantic scores:

$$R = 0.20 \text{ ActionCov} + 0.35 \text{ CFSim} + 0.30 \text{ ConcCov} + 0.15 \text{ LoopCov}. \quad (1)$$

The R measure ensures semantic accuracy along with structural validation of the code structure based on the ground truth diagram. The control flow similarity has the maximum weight because the sequence of operations validates the control flow structure. Concurrency similarity has the second largest weight due to its critical importance, whereas action similarity provides semantic accuracy.

H. Expert Evaluation Procedure

A UML modeling expert also reviewed the results manually in two critical phases: first, during the verification of the dataset, and then during the validation of the output diagrams.

During the construction of the dataset, the reviewer manually verified all the pairs of requirements and diagrams to make sure that the ground-truth PlantUML diagrams accurately represented the described behavior in the natural language requirements. Only diagrams that correctly represented the described behavior were retained as ground truth diagrams.

After the generation process, a total of five representative samples were selected for expert review. For each diagram, the reviewer assigned the weight of the requirement description to the reference diagram, emphasizing behavioral correctness over

TABLE VI: Baseline Model Outputs Compared with Ground-Truth PlantUML Activity Diagram

Ground Truth	Llama 3.1-8B (Baseline)	DeepSeek-R1-8B (Baseline)	Qwen 2.5-Coder-7B (Baseline)
<pre>@startuml start :New transaction received; fork :Check Transaction History; fork again :Check User Location; fork again :Check against Fraud Rules; endfork :Gather results; if (History OR Location OR Rules are 'Suspicious'?) then ([Yes]) :Block Transaction; else ([No]) :Approve Transaction; endif stop @enduml</pre>	<pre>@startuml title Fraud Detection Activity state "Transaction Received" as T state "Check Transaction History" as CT state "Check User Location" as CU state "Check against Fraud Rules" as CFR state "Flag as Suspicious" as FS state "Blocked" as B T -> CT CT -> CU CU -> CFR CFR -> FS FS -> B note right of B : If any check flags the transaction, it is blocked. @enduml</pre>	<pre>@startuml :fraud_detection_system: :Check Transaction History; :Check User Location; :Check against Fraud Rules; :Flag as Suspicious; :Block Transaction; :Blocked Transaction; :Transaction is approved; :Unblock Transaction; @enduml</pre>	<pre>@startuml start :Transaction Received; fork :Check Transaction History; fork again :Check User Location; fork again :Check against Fraud Rules; endfork if (Any check flags as Suspicious?) then ([Yes]) :Block Transaction; else ([No]) :Allow Transaction; endif stop @enduml</pre>

TABLE VII: Comparison of Ground-Truth and Fine-Tuned Model Generated PlantUML Activity Diagrams

Ground Truth	Llama 3.1-8B (SFT + DPO)	DeepSeek-R1-8B (SFT + DPO)	Qwen 2.5-Coder-7B (SFT + DPO)
<pre>@startuml start :New transaction received; fork :Check Transaction History; fork again :Check User Location; fork again :Check against Fraud Rules; endfork :Gather results; if (History OR Location OR Rules are 'Suspicious'?) then ([Yes]) :Block Transaction; else ([No]) :Approve Transaction; endif stop @enduml</pre>	<pre>@startuml start fork :Check Transaction History; fork again :Check User Location; fork again :Check against Fraud Rules; endfork if (History AND Location AND Rules 'Flag as Suspicious'?) then ([Yes]) :Block Transaction; else ([No]) :Approve Transaction; endif stop @enduml</pre>	<pre>@startuml start fork :Run Fraud Detection; fork :Check Transaction History; fork again :Check User Location; fork again :Check against Fraud Rules; endfork if (Any check flags as Suspicious?) then ([Yes]) :Block Transaction; else ([No]) :Approve Transaction; endif stop @enduml</pre>	<pre>@startuml start fork :Check Transaction History; fork again :Check User Location; fork again :Check against Fraud Rules; endfork if (Any check flags as Suspicious?) then ([Yes]) :Block Transaction; else ([No]) :Allow Transaction; endif stop @enduml</pre>

the visual representation of the diagram. The reviewer assessed the presence of required activities, correct action ordering, correct conditional branching, correct loop handling, and correct termination of the workflow process.

The purpose of this manual assessment was to ensure that the structural and semantic evaluation metrics are consistent with human assessments of UML Activity Diagram correctness.

IV. EXPERIMENTAL RESULTS AND ANALYSIS

A. Baseline Diagram Generation

In the baseline setting, for each selected large language model, the model was applied without task-specific fine-tuning. The model was fed only the natural language requirement and produced the corresponding UML Activity Diagram in PlantUML syntax based solely on pre-trained knowledge. As shown in Table VI, this example indicates that baseline outputs may not consistently capture the intended activity-flow semantics, which motivates the subsequent fine-tuning stages.

B. Effect of Model Fine-Tuning (SFT + DPO)

In order to analyze the impact of specialized training, the output diagrams from the process of Supervised Fine-Tuning and Direct Preference Optimization have been analyzed against the baseline results under the same requirement criteria, as shown in Table VII.

It can be seen that the base models were unable to create any valid activity flow diagrams. In their place, the models often created conversational or interaction-type schemas that were devoid of any control flows, leading to structural similarity scores of zero. However, after SFT and DPO were applied, each of the models was able to create valid activity flow diagrams having start/end nodes, correct execution order, and valid decision flows. This suggests that fine-tuning not only increases syntactic

TABLE VIII: Average Structural and Semantic Evaluation Scores of Models Across Training Stages

Model	Setting	ActionCov	CFSim	ConcCov	LoopCov	Overall
Qwen 7B	Baseline	0.668	0.607	0.000	0.000	0.518
Qwen 7B	SFT Only	0.695	0.450	0.400	0.600	0.506
Qwen 7B	SFT + DPO	0.721	0.268	0.728	1.000	0.484
Llama 8B	Baseline	0.094	0.061	0.022	0.015	0.070
Llama 8B	SFT Only	0.820	0.700	0.500	0.050	0.566
Llama 8B	SFT + DPO	0.887	0.826	0.985	0.102	0.825
DeepSeek 8B	Baseline	0.411	0.104	0.000	0.000	0.237
DeepSeek 8B	SFT Only	0.780	0.650	0.300	0.200	0.503
DeepSeek 8B	SFT + DPO	0.832	0.872	0.640	0.690	0.846

validity but also execution semantic understanding, especially when working with concurrent/conditional processes. DeepSeek-R1-8B was the most effective model behaviorally out of the ones evaluated here.

C. Quantitative Metric Comparison

For each of the requirements from the evaluation dataset, the structural and semantic scores of both the base model and the fine-tuned model were determined and averaged. The relative performance results of the studied models, as shown in Table VIII, demonstrate the importance of the fine-tuning approaches adopted. The performance gains are attributed to the SFT model learning syntactic structure patterns and DPO model fine-tuning the reasoning process at an execution level by fixing control flow bugs.

D. Ablation Study: Comparative Impact of SFT and DPO

The performance trends are visualized as shown in Fig. 3. Table VIII shows the average results of structural and semantic similarities for 55 unseen evaluation requirements at each ablation stage. In terms of baselines trained with no task-related supervision, the mapping of the activity logic was particularly difficult, achieving an Overall score of only 0.070 for Llama.

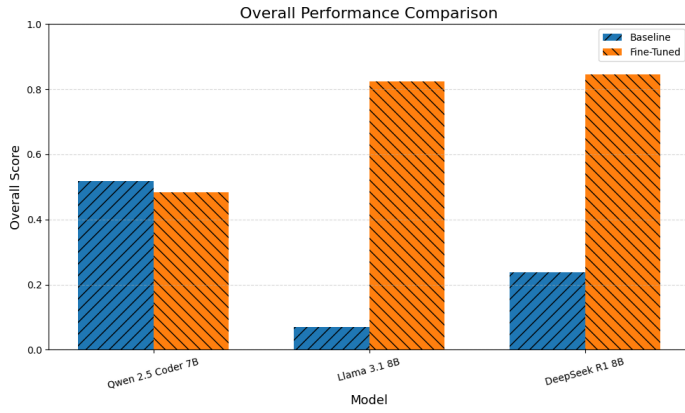


Fig. 3: Overall performance comparison between baseline and fine-tuned models. Fine-tuning using SFT and DPO substantially improves structural similarity across all evaluated models.

This makes our ablation study cleanly delineate between the effects of each stage. First of all, we see that SFT mainly deals with syntactic issues and simple actions' mapping, dramatically boosting 'ActionCov' performance and resulting in a consistent PlantUML template. Yet, even within the SFT-only regime, the models tend to hallucinate synchronization constraints ('ConcCov') and looping conditions. It is the next DPO stage which performs an entirely different job of correcting the model's behavior in an active way by punishing structural errors via preference rewards, thus significantly boosting the overall score of 0.566 to 0.825 for Llama.

Once all of the capabilities of SFT+DPO were harnessed, both Llama and DeepSeek verified their findings regarding structural activities and achieved optimal valid starting nodes, branching, and concurrent sequences. The high degree of general reasoning exhibited by Llama 3.1-8B proved effective in adjusting itself to the DPO framework and translating natural language instructions into appropriate branching decisions. In DeepSeek-R1-8B, which was specifically designed for logical reasoning, concurrent states synchronization became easy, resulting in an Overall Score of 0.846.

In the case of the fine-tuned Qwen 2.5-Coder-7B model, the Overall score dropped from 0.518 to 0.484. While there were improvements in the concurrency and loop activities (ConcCov: 0.728; LoopCov: 1.000), the control flow similarity score showed an alarming degradation (CFSim reduced from 0.607 to 0.268). It can be inferred that, owing to the fact that Qwen 2.5-Coder was fine-tuned for robust code syntax structures, the model got overfitted with specific preference criteria during the DPO procedure, resulting in catastrophic forgetting. In doing so, the Qwen 2.5-Coder lost its ability to map generalized control flows and focused on isolated edge behaviors.

E. Cross-Dataset Validation

Moreover, to evaluate the reliability of the suggested approach, the performance of the optimal DeepSeek 8B architecture trained using SFT and DPO approaches was examined against external requirements. These external requirements were obtained from publicly accessible workflow descriptions. This external data was carefully chosen to incorporate various types of workflows with differing levels of control flow complexity. The external dataset comprises 20 requirements taken from fields like e-commerce, banking, and health care. True PlantUML diagrams have been created manually for these requirements. As shown in Table IX, the accuracy attained by the model on this external dataset was 0.813, which is comparable with internal results, i.e., 0.846. It

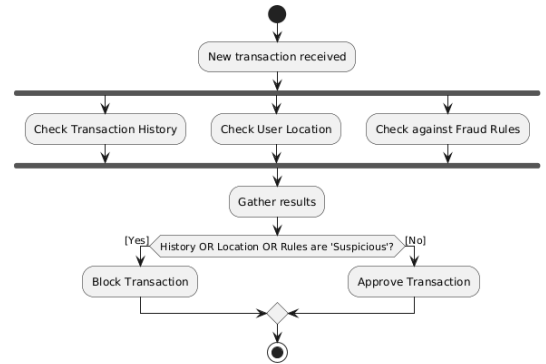
clearly indicates the robustness of our framework for requirements from different fields. This minor decline in performance has been explained by the difference in fields for the external requirements. Nevertheless, the size of this external dataset has limitations, which makes large-scale validation necessary.

TABLE IX: Cross-Dataset Validation Results for DeepSeek 8B (SFT + DPO)

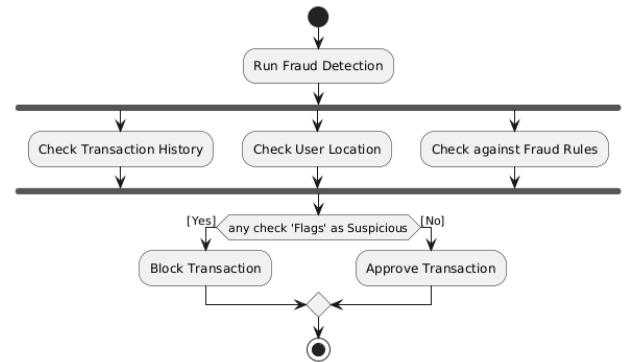
Metric	Internal Dataset	External Dataset
ActionCov	0.832	0.808
CFSim	0.872	0.842
ConcCov	0.640	0.605
LoopCov	0.690	0.658
Overall Score	0.846	0.813

F. Qualitative Diagram Comparison

In the ground-truth diagram, as shown in Fig. 4, it is clear that there are parallel activities which synchronize before making any decision. This synchronization process is captured by the generated diagram accurately. However, the baseline models tend to produce diagrams where activities follow a linear sequence, and hence, no decision-making happens. Similar results were reported in other application areas like e-commerce and healthcare processes. For instance, in the case of order processing with condition-based payment verification and parallel checking of stock, fine-tuned models precisely modeled forking/joining and conditionality, implying greater structural and semantic accuracy. This supports the claim that SFT and DPO training allows the models to understand structural dependencies at the execution level as opposed to creating a linearized representation of the program structure.



(a) Ground-Truth Diagram



(b) Generated Diagram (Fine-Tuned DeepSeek Model)

Fig. 4: Qualitative comparison between reference and generated UML Activity Diagrams.

G. Human Expert Validation

To determine if the selected evaluation criteria correspond to human understanding of behavioral correctness, a manual analysis was performed. The produced activity diagrams were analyzed by an experienced UML modeler who determined whether the behavioral correctness of the diagram accurately matched the required behavior. Behavioral correctness encompasses the inclusion of necessary actions, proper sequencing, appropriate decision-making, and correct workflow termination. The expert evaluations were compared against the automated metrics, and as shown in Table X, there is a high degree of concordance between the automated scores and expert evaluations.

Requirement: A user wants to rent a car. They select a car and choose rental dates. The system checks whether the user is over 25 years old. If not, an underage fee is added; otherwise, no fee is applied. Finally, the system calculates the total rental price.

TABLE X: Automatic Metric Scores vs. Human Expert Evaluation

Model	ActionCov	CFSim	ConcCov	Overall
Fine-Tuned Llama 3.1-8B	0.91	0.86	0.95	0.89
Fine-Tuned DeepSeek-R1-8B	0.93	0.91	0.96	0.92
Fine-Tuned Qwen 2.5-Coder-7B	0.78	0.10	0.70	0.36
Human Evaluation	0.94	0.92	0.96	0.93

For instance, in the above example, the results are very consistent with the qualitative judgment of the expert. The DeepSeek-R1 model received the maximum score of 0.92, which is close to the expert score of 0.93. Inconsistent behavioral characteristics include the wrong decision-making algorithm and control flow similarity for Qwen. This test is not a statistical verification process. It requires a larger-scale human verification process to confirm the validity of metrics.

V. CONCLUSION AND FUTURE WORK

In this study, we develop a method that uses fine-tuned open-source large language models to automatically generate structurally and semantically correct UML activity diagrams based on natural language requirements. By combining Supervised Fine-Tuning (SFT) and Direct Preference Optimization (DPO), the proposed method differentiates syntactic training and behavioral reasoning, thereby facilitating more accurate modeling of execution procedures and control flow. We propose a structure-aware semantic evaluation framework for evaluating the generated diagrams through measures like Action Coverage, Control-Flow Similarity, Concurrency Coverage, and Loop Coverage. Our experimental findings reveal that the generated diagrams using baselines exhibit deficiencies in completeness and consistency, while SFT enhances structural soundness and DPO boosts control-flow accuracy, concurrency management, and loop modeling. Overall, DeepSeek-R1-8B outperforms other large language models in terms of generation quality in various evaluation settings. However, there are still several restrictions due to which it is necessary to further develop this framework to make it more generalized, robust, and scalable, which include the absence of large publicly available requirement-PlantUML datasets as well as evaluation in cases of industrial workflows that include more complicated control structures. As part of future work, the dataset should be extended for industrial applications and use more advanced PlantUML syntax with exceptions, swimlanes, and hierarchy of activities. Other types of UML diagrams like use-case, state, and sequence diagrams could be considered in future research. It would also be possible to implement automatic validation, learning feedback features, and

conduct large-scale human evaluation studies to provide more generalized solutions. Possible practical application scenarios include automation of software design and educational assessment in software engineering.

REFERENCES

- [1] O. S. Dawood and A.-E.-K. Sahraoui, "From requirements engineering to uml using natural language processing - survey study," *European Journal of Engineering Research and Science (EJERS)*, vol. 2, no. 1, pp. 44–49, 2017.
- [2] A. M. Alashqar, "Automatic generation of uml diagrams from scenario-based user requirements," *Jordanian Journal of Computers and Information Technology (JJCIT)*, vol. 7, no. 2, pp. 180–196, 2021.
- [3] B. Hnatkowska and M. Cebinka, "Activity diagram generation based on use-case textual specification," *Computing and Informatics*, vol. 40, no. 4, pp. 772–795, 2021.
- [4] C. R. Narawita and K. Vidanage, "Uml generator – use case and class diagram generation from text requirements," *International Journal on Advances in ICT for Emerging Regions (ICTer)*, vol. 10, no. 1, 2017.
- [5] S. Nasiri, A. Adadi, and M. Lahmer, "Automatic generation of business process models from user stories," *International Journal of Electrical and Computer Engineering (IJECE)*, vol. 13, no. 1, pp. 809–822, 2023.
- [6] J. Wei, C. Tan, Q. Chen, G. Wu, S. Li, Z. Gao, L. Sun, B. Yu, and R. Guo, "From words to structured visuals: A benchmark and framework for text-to-diagram generation and editing," *arXiv preprint arXiv:2411.11916*, 2024.
- [7] N. G. S. Krishnan, S. Ambadi, and M. G. Thushara, "Comparative analysis of large language models for automated use case diagram generation," in *Proceedings of the 3rd International Conference on Futuristic Technology (INCOFT 2025)*. SCITEPRESS, 2025, pp. 465–471.
- [8] A. Ferrari, S. Abualhaija, and C. Arora, "Model generation with llms: From requirements to uml sequence diagrams," *arXiv preprint arXiv:2404.06371*, 2024, iEEE International Requirements Engineering Conference (RE'24).
- [9] P. Khamsepour, M. Cole, I. Ashraf, S. Puri, M. Sabetzadeh, and S. Nejati, "The impact of critique on llm-based model generation from natural language: The case of activity diagrams," *arXiv preprint arXiv:2509.03463*, 2025.
- [10] K. Ahmed, O. Wei, J. Song, B. Chen, and B. Zheng, "Mecet: Behavioral model correctness evaluation using large language models," *arXiv preprint arXiv:2508.00630*, 2025.
- [11] N. Boualia, M. Gerhold, T. U. Rehman, and F. Ahmed, "Toward automated uml diagram assessment: Comparing llm-generated scores with teaching assistants," in *Proceedings of the 17th International Conference on Computer Supported Education (CSEDU 2025)*, vol. 1. SCITEPRESS, 2025, pp. 158–169.
- [12] A. A. Fadhil and A. H. Albayati, "Automatic evaluation of the student's activity diagrams in the educational process," *Ingénierie des Systèmes d'Information*, vol. 30, no. 8, pp. 2149–2156, 2025.
- [13] A. Bates, R. Vavricka, S. Carleton, R. Shao, and C. Pan, "Unified modeling language code generation from diagram images using multimodal large language models," *Machine Learning with Applications*, vol. 20, p. 100660, 2025.
- [14] R. P. Nair, M. G. Thushara, and V. Sugumaran, "Fine-tuned llms versus rule-based nlp for uml diagram generation: An educational evaluation," *IEEE Access*, 2025.
- [15] R. P. Nair and M. G. Thushara, "Nl2code: A hybrid nlp and model-driven framework for automated code generation from natural language and uml," in *2025 IEEE International Students Conference on Electrical, Electronics and Computer Science (SCECS)*. IEEE, 2025.
- [16] D. Rouabhia and I. Hadjadi, "Behavioral augmentation of uml class diagrams: An empirical study of large language models for method generation," *arXiv preprint arXiv:2506.00788*, 2025.
- [17] C. Wang, B. Wang, P. Liang, and J. Liang, "Assessing uml diagrams by chatgpt: Implications for education," *arXiv preprint arXiv:2412.17200*, 2025, preprint submitted to Elsevier.
- [18] F. Siewe and G. M. Ngounou, "On the execution and runtime verification of uml activity diagrams," *Software*, vol. 4, no. 1, p. 4, 2025.
- [19] J. Torcal, V. Moreno, J. Llorens, and A. Granados, "Creating and validating a ground truth dataset of unified modeling language diagrams using deep learning techniques," *Applied Sciences*, vol. 14, no. 23, p. 10873, 2024.
- [20] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, 2024.
- [21] V. S. N. L. Yarramallu, R. J. Gangireddy, L. N. Manne, and V. G., "Hybridcode: A dual-database framework for intelligent codebase analysis and article generation," in *2025 5th Asian Conference on Innovation in Technology (ASIANCON)*, 2025.
- [22] G. Krishna, A. Sabu, A. A. and V. G., "A generative and conversational ai system for after-school learning," in *2025 5th Asian Conference on Innovation in Technology (ASIANCON)*, 2025.
- [23] A. S. A. G. Krishnan, and V. G., "Leveraging technology to empower millet farmers: A retrieval-augmented generation approach with large language models," in *2024 5th IEEE Global Conference for Advancement in Technology (GCAT)*, 2024.